Document 07: API OpenAPI Specification

CodePerfect Auditor — Complete REST API Reference

Project: CodePerfect Auditor | **Version:** 1.0 | **Date:** 31-03-2026 **Submitted To:** Virtusa Hackathon - Jatayu | **Institution:** Saveetha Engineering College

---

Overview

CodePerfect Auditor exposes a fully documented **FastAPI REST API** at `http://localhost:8000`. FastAPI auto-generates an interactive Swagger UI at `GET /docs` and an OpenAPI 3.0 JSON schema at `GET /openapi.json`.

Base URL: `http://localhost:8000/api/v1` **Auth:** Supabase service-role key via `Authorization: Bearer <token>` header **Format:** JSON request/response bodies. PDF uploads use `multipart/form-data`.

Application Bootstrap — `main.py`

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from contextlib import asynccontextmanager
from database import get_client, close_client
from routes import icd, health, parse, code, cases, analytics, claims, payers

@asynccontextmanager
async def lifespan(app: FastAPI):
    """
    ASGI lifespan context — runs startup and shutdown logic.
    On startup: warm up the async HTTP client and verify Supabase connectivity.
    On shutdown: close the HTTP client to release TCP connections gracefully.
    """
    try:
        client = await get_client()
        resp = await client.get("/icd_codes", params={"select": "code", "limit":
"1"})
        if resp.status_code == 200:
            print("✅ Supabase REST API connected")
    except Exception as e:
        print(f"⚠️  Supabase connection warning: {e}")
    yield
    await close_client()   # Release the shared HTTP connection pool on shutdown

app = FastAPI(
    title="CodePerfect Auditor API",
    description="Revenue Integrity Engine — Agentic ICD-10/11 Coding, Audit &
Revenue Intelligence",
```

```
    version="1.0.0",
    lifespan=lifespan,
)

# CORS: allow requests only from the trusted Next.js frontend origins
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000", "http://localhost:3001",
                   "http://127.0.0.1:3000", "http://127.0.0.1:3001"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Register all route modules under the /api/v1 prefix
app.include_router(health.router)                          # No prefix (GET /health)
app.include_router(icd.router,       prefix="/api/v1")  # GET /api/v1/icd/*
app.include_router(parse.router,     prefix="/api/v1")  # POST /api/v1/parse/*
app.include_router(code.router,      prefix="/api/v1")  # POST /api/v1/code/*
app.include_router(cases.router,     prefix="/api/v1")  # GET  /api/v1/cases/*
app.include_router(analytics.router, prefix="/api/v1")  # GET  /api/v1/analytics/*
app.include_router(claims.router,    prefix="/api/v1")  # POST/GET /api/v1/claims/*
app.include_router(payers.router,    prefix="/api/v1")  # GET/PUT /api/v1/payers/*
```

Endpoint Group 1: Health Check

GET /health

```
GET http://localhost:8000/health

Response 200:
{
  "status": "healthy",
  "service": "CodePerfect Auditor API",
  "version": "1.0.0"
}
```

Purpose: Used by Docker health checks, load balancers, and monitoring systems (Uptime Robot, AWS ALB) to verify the server is running. Returns instantly with no database query — ensuring the health probe never times out due to DB latency.

Endpoint Group 2: ICD Code Lookup

GET /api/v1/icd/search?query={term}&limit={n}

```
GET /api/v1/icd/search?query=diabetes&limit=10

Response 200:
[
  {
    "code": "E11.22",
    "description": "Type 2 diabetes mellitus with diabetic chronic kidney disease,
stage 3",
    "chapter": "Endocrine, nutritional and metabolic diseases",
    "is_billable": true,
    "is_cc": false,
    "is_mcc": true,
    "base_reimbursement": 5000.00,
    "icd_version": "ICD-10-CM-2024"
  },
  ...
]
```

Purpose: Powers the autocomplete search box in the hospital coding workspace. Uses a PostgREST `ilike` filter on `description` and `code` columns for fast text-based lookup without embedding.

Endpoint Group 3: AI Coding Pipeline

POST /api/v1/code/run — Text Input Pipeline

```
POST /api/v1/code/run
Content-Type: application/json

Request Body:
{
  "raw_text": "Patient: Male, 62 years old. Admitted with NSTEMI and CRF stage 3.
              History of Type 2 DM, HbA1c 9.2%. No retinopathy noted.",
  "session_id": "550e8400-e29b-41d4-a716-446655440000",   // Optional — generated
if absent
  "human_icd_code": "E11.9",                               // Optional — for audit
comparison
  "org_id": "de305d54-75b4-431b-adb2-eb6b9e546014"        // Optional — for CPT
multiplier
}
```

Response 200:


```
{
  "session_id": "550e8400-e29b-41d4-a716-446655440000",
  "final_icd_code": "I21.4",
  "confidence_score": 0.98,
  "mapping_path": "gold_standard_override",
  "resolved_snomed_code": "401303003",
  "icd_codes": [
    {
      "code": "I21.4",
      "description": "Non-ST elevation myocardial infarction",
      "role": "primary",
      "final_score": 0.98,
      "is_mcc": true,
      "is_cc": false,
      "base_reimbursement": 12000.00,
      "rationale": "MCC — Major Complication/Comorbidity; Gold Standard keyword
'NSTEMI' matched"
    },
    {
      "code": "E11.22",
      "description": "Type 2 diabetes mellitus with diabetic chronic kidney
disease",
      "role": "secondary",
      "final_score": 0.71,
      "is_mcc": false,
      "is_cc": true,
      "base_reimbursement": 5000.00,
      "rationale": "CC — Complication/Comorbidity; exact SNOMED match"
    }
  ],
  "cpt_codes": [
    {
      "code": "93458",
      "description": "Catheterization, left heart with coronary angiography",
      "base_price": 2300.00,
      "multiplier": 1.5,
      "gross_charge": 3450.00,
      "confidence": 0.87
    }
  ],
  "discrepancy_type": "SPECIFICITY_IMPROVEMENT",
  "financial_delta": 6100.00,
  "drg_flag": null,
  "risk_score": 0.18,
  "risk_label": "LOW",
  "financial_summary": {
    "total_estimated_revenue": 3450.00,
    "pricing_multiplier": 1.5,
    "line_items": [...]
  },
  "fhir_condition": { "resourceType": "Condition", "code": {...}, ... },
  "patient_name": "John Smith",
  "patient_dob": "1963-07-15",
  "document_source": "text_input",
  "ocr_used": false,
```

```
  "extraction_metadata": {
    "model": "llama-3.3-70b-versatile",
    "llm_version": "2024-12",
    "icd_version": "ICD-10-CM-2024",
    "snomed_version": "SNOMED-CT-2024"
  }
}
```

`POST /api/v1/code/run-pdf` — PDF Upload Pipeline

```
POST /api/v1/code/run-pdf
Content-Type: multipart/form-data

Form Fields:
  file:           <binary PDF file>    // Max 20MB
  session_id:     "uuid-string"        // Optional
  human_icd_code: "E11.9"             // Optional
  org_id:         "uuid-string"        // Optional

Response 200: Same as /code/run above.
              Additional fields:
              "document_source": "pdf_upload"
              "ocr_used": true   // If Tesseract OCR was triggered (scanned PDF)
```

Pipeline flow: `doc_processing_node` → `clinical_extraction_agent` → `snomed_resolver` → `snomed_icd_mapper` → [`icd_embedding` if needed] → `icd_decision` → `audit_comparison` → `risk_scoring` → `financial_calculator`.

Endpoint Group 4: Case History

`GET /api/v1/cases` — Paginated Case List

```
GET /api/v1/cases?page=1&page_size=20&risk_label=HIGH&document_source=pdf_upload

Query Parameters:
  page            integer  (default: 1)          Page number
  page_size       integer  (default: 20, max:100) Rows per page
  risk_label      string   (optional)             LOW | MEDIUM | HIGH
  document_source string   (optional)             text_input | pdf_upload
  branch_id       UUID     (optional)             Filter by hospital branch
```

```
Response 200:
{
  "cases": [
    {
      "result_id": "uuid",
      "session_id": "uuid",
      "ai_icd_code": "E11.22",
      "human_icd_code": "E11.9",
      "discrepancy_type": "SPECIFICITY_IMPROVEMENT",
      "financial_delta": 3200.00,
      "risk_score": 0.18,
      "risk_label": "LOW",
      "confidence_score": 0.87,
      "drg_flag": null,
      "document_source": "pdf_upload",
      "ocr_used": false,
      "text_snippet": "Patient: Male, 62 years old. Admitted with...",
      "created_at": "2026-03-31T00:15:44.321Z"
    }
  ],
  "total": 247,
  "page": 1,
  "page_size": 20,
  "total_pages": 13
}
```

`GET /api/v1/cases/stats` — Aggregate KPIs

```
GET /api/v1/cases/stats

Response 200:
{
  "total_cases": 247,
  "total_revenue_recovered": 184250.50,
  "high_risk_count": 18,
  "accuracy_rate": 76.3
}
```

`GET /api/v1/cases/{session_id}` — Single Case Detail

```
GET /api/v1/cases/550e8400-e29b-41d4-a716-446655440000

Response 200: Full CodeResponse-compatible object
              (Same schema as POST /code/run response)
```

```
              Allows the Cases History page to re-render the full ResultsPanel.

Response 404: { "detail": "Case '550e...' not found." }
```

Endpoint Group 5: Analytics Dashboard

GET /api/v1/analytics/overview

```
GET /api/v1/analytics/overview

Response 200:
{
  "total_cases": 247,
  "total_revenue_recovered": 184250.50,
  "avg_confidence": 83.4,
  "high_risk_rate": 7.3,
  "risk_distribution": { "LOW": 183, "MEDIUM": 46, "HIGH": 18 },
  "source_distribution": { "text_input": 162, "pdf_upload": 85 },
  "trend": [
    { "date": "2026-03-01", "cases": 8, "revenue": 5400.00 },
    { "date": "2026-03-02", "cases": 12, "revenue": 8200.00 },
    ...   // 30 data points total (one per day)
  ]
}
```

GET /api/v1/analytics/top-codes

```
GET /api/v1/analytics/top-codes

Response 200:
{
  "codes": [
    {
      "code": "E11.22",
      "count": 47,
      "avg_revenue": 3840.50,
      "avg_risk": 0.21,
      "top_discrepancy": "SPECIFICITY_IMPROVEMENT"
    },
    ...  // Top 10 codes by frequency
  ]
}
```

GET /api/v1/analytics/discrepancy-breakdown

```
GET /api/v1/analytics/discrepancy-breakdown

Response 200:
[
  { "type": "NO_COMPARISON",           "label": "— No Comparison",   "count": 121
},
  { "type": "EXACT_MATCH",             "label": "✓ Exact Match",      "count": 67
},
  { "type": "SPECIFICITY_IMPROVEMENT", "label": "↑ Specificity",      "count": 38
},
  { "type": "CODE_DIVERGENCE",         "label": "⚠ Diverged",          "count": 12
},
  { "type": "OVERCODING",              "label": "⬆ Overcode",          "count": 6
},
  { "type": "UNSUPPORTED_CODE",        "label": "✗ Unsupported",       "count": 3 }
]
```

Endpoint Group 6: Claims Lifecycle

POST /api/v1/claims/submit

```
POST /api/v1/claims/submit
Content-Type: application/json

{
  "session_id":          "550e8400-e29b-41d4-a716-446655440000",
  "organization_id":     "de305d54-75b4-431b-adb2-eb6b9e546014",
  "payer_id":            "a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11",
  "patient_name":        "John Smith",
  "patient_dob":         "1963-07-15",
  "total_billed_amount": 3450.00,
  "claim_data": {
    "icd_codes":        [...],
    "cpt_codes":        [...],
    "financial_summary": {...}
  },
  "submission_notes": "NSTEMI admission — urgent processing requested"
}

Response 201:
{
  "claim_id": "uuid-of-new-claim",
```

```
  "status": "SUBMITTED",
  "payer_gate_report": {
    "decision": "MANUAL_REVIEW",
    "reasons": ["risk_score 0.52 exceeds payer threshold 0.40"]
  },
  "message": "Claim submitted successfully."
}

Response 400:
{ "detail": "A claim has already been submitted for this session." }
```

POST /api/v1/claims/{claim_id}/adjudicate

```
POST /api/v1/claims/uuid/adjudicate
Content-Type: application/json

{
  "status":              "PAID",                    // PAID | DENIED |
PARTIALLY_PAID
  "total_allowed_amount": 2875.00,
  "total_paid_amount":    2300.00,
  "patient_responsibility": 575.00,
  "denial_reason":         null                     // Required if status = DENIED
}

Response 200:
{
  "claim_id": "uuid",
  "status": "PAID",
  "total_paid_amount": 2300.00,
  "message": "Claim adjudicated successfully."
}
```

GET /api/v1/claims/{claim_id}/edi-export

```
GET /api/v1/claims/uuid/edi-export?type=837

Response 200:
Content-Type: text/plain
Content-Disposition: attachment; filename="claim_uuid_837.edi"

ISA*00*          *00*          *ZZ*INTGRNX01      *ZZ*GLBLHLTH
*260331*0030*^*00501*000000001*0*T*:~
GS*HC*INTGRNX01*GLBLHLTH*20260331*0030*1*X*005010X222A1~
ST*837*0001*005010X222A1~
```

```
BHT*0019*00*9B3862E1E4E742*20260331*0030*CH~
...
```

Endpoint Group 7: Payer Configuration

GET /api/v1/payers/{payer_id}/settings

```
GET /api/v1/payers/a0eebc99/settings

Response 200:
{
  "payer_id": "a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11",
  "auto_approve_enabled": true,
  "auto_approve_confidence_min": 0.80,
  "auto_approve_max_risk": 0.40,
  "auto_approve_requires_patient_dob": true,
  "auto_approve_requires_patient_sex": false,
  "auto_approve_payer_responsibility_pct": 0.80,
  "accepted_icd_versions": ["ICD-10", "ICD-11"],
  "auto_approve_custom_rules": [
    {
      "rule_type": "max_amount",
      "label": "Claims over $50,000 require manual review",
      "threshold": 50000.0
    },
    {
      "rule_type": "exclude_cpt_prefix",
      "label": "Block cardiac surgery codes",
      "code_prefix": "33"
    }
  ]
}
```

PUT /api/v1/payers/{payer_id}/settings

```
PUT /api/v1/payers/a0eebc99/settings
Content-Type: application/json

{
  "auto_approve_enabled": true,
  "auto_approve_confidence_min": 0.85,
  "auto_approve_max_risk": 0.35,
  "auto_approve_custom_rules": [...]
}
```

```
Response 200:
{ "message": "Settings updated successfully." }
```

Complete API Summary Table

Method	Path	Description	Auth
GET	`/health`	Server health probe	None
GET	`/api/v1/icd/search`	ICD code text search	Anon key
POST	`/api/v1/code/run`	Run AI pipeline (text)	Anon key
POST	`/api/v1/code/run-pdf`	Run AI pipeline (PDF)	Anon key
GET	`/api/v1/cases`	Paginated case history	Service key
GET	`/api/v1/cases/stats`	Aggregate KPI stats	Service key
GET	`/api/v1/cases/{session_id}`	Single case detail	Service key
GET	`/api/v1/analytics/overview`	Dashboard KPIs + trend	Service key
GET	`/api/v1/analytics/top-codes`	Top 10 codes	Service key
GET	`/api/v1/analytics/discrepancy-breakdown`	Discrepancy donut	Service key
POST	`/api/v1/claims/submit`	Submit claim to payer	Service key
POST	`/api/v1/claims/{id}/adjudicate`	Payer adjudication	Service key

Method	Path	Description	Auth
GET	`/api/v1/claims/{id}/edi-export`	Download EDI 837/835	Service key
GET	`/api/v1/payers/{id}/settings`	Read payer config	Service key
PUT	`/api/v1/payers/{id}/settings`	Update payer config	Service key

Interactive Docs: `http://localhost:8000/docs` (Swagger UI) **OpenAPI JSON:** `http://localhost:8000/openapi.json`

CodePerfect Auditor | Virtusa Hackathon 2026 | Saveetha Engineering College